Step 1: Import Libraries and Setup

```
import numpy as np
import pandas as pd

# Data fetching
import yfinance as yf

# Install pykalman if not already installed
!pip install pykalman

# Kalman Filter
from pykalman import KalmanFilter

# Machine Learning
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# Statistical tests
from statsmodels.tsa.stattools import adfuller
from scipy import stats

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Settings
%matplotlib inline
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
np.random.seed(42)

import warnings
warnings.filterwarnings('ignore')

print("✓ All libraries imported successfully!")
```

```
Collecting pykalman
  Downloading pykalman-0.11.0-py2.py3-none-any.whl.metadata (9.7 kB)
Requirement already satisfied: numpy<3 in /usr/local/lib/python3.12/dist-packages (from pykalman) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from pykalman) (25.0)
Collecting scikit-base<0.14.0 (from pykalman)
  Downloading scikit_base-0.13.0-py3-none-any.whl.metadata (8.8 kB)
Requirement already satisfied: scipy<2.0.0 in /usr/local/lib/python3.12/dist-packages (from pykalman) (1.16.3)
Downloading pykalman-0.11.0-py2.py3-none-any.whl (249 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 249.4/249.4 kB 10.0 MB/s eta 0:00:00
Downloading scikit_base-0.13.0-py3-none-any.whl (151 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 151.5/151.5 kB 12.5 MB/s eta 0:00:00
Installing collected packages: scikit-base, pykalman
Successfully installed pykalman-0.11.0 scikit-base-0.13.0
✓ All libraries imported successfully!
```

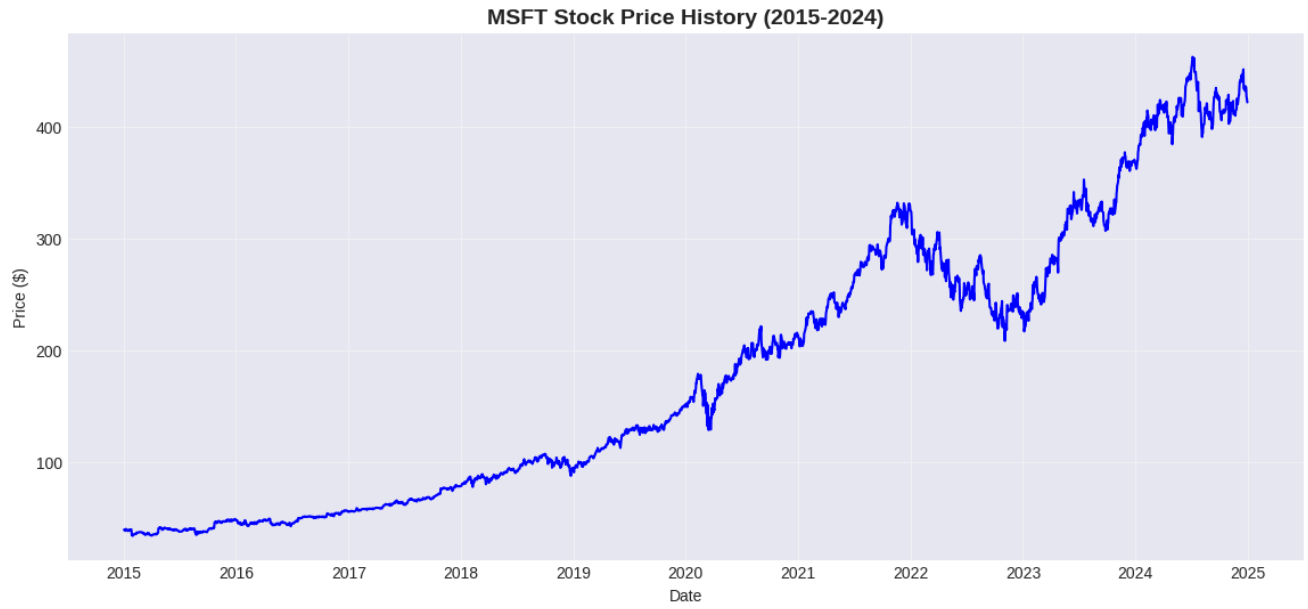
Step 2: Fetch MSFT Historical Data (2015-2024)

```
# Download MSFT data from Yahoo Finance
ticker = yf.Ticker("MSFT")
df = ticker.history(start='2015-01-01', end='2024-12-31')

# Remove timezone info
if df.index.tz is not None:
    df.index = df.index.tz_localize(None)

# Basic info
print(f"Data fetched: {len(df)} trading days")
print(f"Date range: {df.index[0].date()} to {df.index[-1].date()}")
print(f"\nColumns: {list(df.columns)}")

# Display first few rows
df.head()
```

```
Data fetched: 2515 trading days
Date range: 2015-01-02 to 2024-12-30

Columns: ['Open', 'High', 'Low', 'Close', 'Volume', 'Dividends', 'Stock Splits']
```

Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
2015-01-02	39.773209	40.421035	39.670921	39.858448	27913900	0.0	0.0
2015-01-05	39.526010	39.832877	39.423723	39.491917	39673900	0.0	0.0
2015-01-06	39.534536	39.849925	38.818516	38.912281	36447900	0.0	0.0
2015-01-07	39.193572	39.602726	38.775896	39.406673	29114100	0.0	0.0
2015-01-08	39.849928	40.702333	39.824357	40.565948	29645200	0.0	0.0

```
# Visualize price history
plt.figure(figsize=(14, 6))
plt.plot(df.index, df['Close'], linewidth=1.5, color='blue')
plt.title('MSFT Stock Price History (2015-2024)', fontsize=14, fontweight='bold')
plt.xlabel('Date')
plt.ylabel('Price ($)')
plt.grid(True, alpha=0.3)
plt.show()

print(f"Starting price: ${df['Close'].iloc[0]:.2f}")
print(f"Ending price: ${df['Close'].iloc[-1]:.2f}")
print(f"Total return: {(df['Close'].iloc[-1]/df['Close'].iloc[0] - 1)*100:.2f}%")
```




```
Starting price: $39.86
Ending price: $421.72
Total return: 958.04%
```

Step 3: Stationarity Testing with ADF

Before applying Kalman Filters, we should test if our time series is stationary.

- **Null Hypothesis**: Series has a unit root (non-stationary)
- **Alternative**: Series is stationary
- If p-value < 0.05, we reject null hypothesis (series is stationary)

```
# Test 1: ADF test on price levels
print("="*70)
print("ADF Test on Price Levels")
```

```
print("="*70)

result = adfuller(df['Close'].dropna(), autolag='AIC')

print(f'ADF Statistic: {result[0]:.6f}')
print(f'p-value: {result[1]:.6f}')
print(f'Number of lags used: {result[2]}')
print(f'Number of observations: {result[3]}')
print('\nCritical Values:')
for key, value in result[4].items():
    print(f'   {key}: {value:.3f}')

# Interpretation
if result[1] > 0.05:
    print("\n Result: Price series is NON-STATIONARY (p > 0.05)")
    print("   We cannot reject the null hypothesis.")
    print("   This is expected for stock prices!")
else:
    print("\n✓ Result: Price series is STATIONARY (p < 0.05)")
    print("   We reject the null hypothesis.")
```

```
======================================================================
ADF Test on Price Levels
======================================================================
ADF Statistic: 0.759779
p-value: 0.990958
Number of lags used: 26
Number of observations: 2488

Critical Values:
   1%: -3.433
   5%: -2.863
   10%: -2.567

 Result: Price series is NON-STATIONARY (p > 0.05)
   We cannot reject the null hypothesis.
   This is expected for stock prices!
```

```
# Test 2: ADF test on returns (should be stationary)
print("="*70)
print("ADF Test on Returns")
print("="*70)

returns = df['Close'].pct_change().dropna()
result = adfuller(returns, autolag='AIC')

print(f'ADF Statistic: {result[0]:.6f}')
print(f'p-value: {result[1]:.6f}')
print(f'Number of lags used: {result[2]}')
print(f'Number of observations: {result[3]}')
print('\nCritical Values:')
for key, value in result[4].items():
    print(f'   {key}: {value:.3f}')

# Interpretation
if result[1] < 0.05:
    print("\n✓ Result: Returns series is STATIONARY (p < 0.05)")
    print("   We reject the null hypothesis.")
    print("   Good! We can use returns for modeling.")
else:
    print("\n⚠️ Result: Returns series is NON-STATIONARY (p > 0.05)")
```

```
======================================================================
ADF Test on Returns
======================================================================
ADF Statistic: -17.367391
p-value: 0.000000
Number of lags used: 8
Number of observations: 2505

Critical Values:
   1%: -3.433
   5%: -2.863
   10%: -2.567

✓ Result: Returns series is STATIONARY (p < 0.05)
   We reject the null hypothesis.
   Good! We can use returns for modeling.
```

Step 4: Feature Engineering

Create technical indicators and features for the model.

```
# Calculate returns
df['Returns'] = df['Close'].pct_change()
df['Log_Returns'] = np.log(df['Close'] / df['Close'].shift(1))

# Moving Averages
df['MA_5'] = df['Close'].rolling(window=5).mean()
df['MA_20'] = df['Close'].rolling(window=20).mean()
df['MA_60'] = df['Close'].rolling(window=60).mean()

# Moving Average Ratios
df['MA_Ratio_5_20'] = df['MA_5'] / df['MA_20']
df['MA_Ratio_20_60'] = df['MA_20'] / df['MA_60']

# Lagged Returns
for lag in [1, 2, 3, 5, 10]:
    df[f'Returns_Lag_{lag}'] = df['Returns'].shift(lag)

# Rate of Change (ROC)
for period in [5, 10, 20]:
    df[f'ROC_{period}'] = ((df['Close'] - df['Close'].shift(period)) / df['Close'].shift(period)) * 100

# Volatility
df['Volatility_10'] = df['Returns'].rolling(window=10).std()
df['Volatility_20'] = df['Returns'].rolling(window=20).std()
df['Volatility_60'] = df['Returns'].rolling(window=60).std()

# Volume features
df['Volume_MA_20'] = df['Volume'].rolling(window=20).mean()
df['Volume_Ratio'] = df['Volume'] / df['Volume_MA_20']

# Momentum
df['Momentum_5'] = df['Close'] / df['Close'].shift(5) - 1
df['Momentum_20'] = df['Close'] / df['Close'].shift(20) - 1

# Bollinger Bands
df['BB_Middle'] = df['Close'].rolling(window=20).mean()
bb_std = df['Close'].rolling(window=20).std()
df['BB_Upper'] = df['BB_Middle'] + (2 * bb_std)
df['BB_Lower'] = df['BB_Middle'] - (2 * bb_std)
df['BB_Position'] = (df['Close'] - df['BB_Lower']) / (df['BB_Upper'] - df['BB_Lower'])

# RSI
delta = df['Close'].diff()
gain = (delta.where(delta > 0, 0)).rolling(window=14).mean()
loss = (-delta.where(delta < 0, 0)).rolling(window=14).mean()
rs = gain / loss
df['RSI'] = 100 - (100 / (1 + rs))

# Drop NaN rows
df_clean = df.dropna().copy()

print(f"Features created: {len(df_clean.columns)} total columns")
print(f"Clean dataset: {len(df_clean)} rows (dropped {len(df) - len(df_clean)} NaN rows)")
print(f"\nNew feature columns:")
new_cols = [col for col in df_clean.columns if col not in ['Open', 'High', 'Low', 'Close', 'Volume', 'Dividends', 'Stock Splits
for i, col in enumerate(new_cols, 1):
    print(f"{i:2d}. {col}")
```

```
Features created: 34 total columns
Clean dataset: 2455 rows (dropped 60 NaN rows)

New feature columns:
 1. Returns
 2. Log_Returns
 3. MA_5
 4. MA_20
 5. MA_60
 6. MA_Ratio_5_20
 7. MA_Ratio_20_60
 8. Returns_Lag_1
 9. Returns_Lag_2
10. Returns_Lag_3
11. Returns_Lag_5
```

```
12. Returns_Lag_10
13. ROC_5
14. ROC_10
15. ROC_20
16. Volatility_10
17. Volatility_20
18. Volatility_60
19. Volume_MA_20
20. Volume_Ratio
21. Momentum_5
22. Momentum_20
23. BB_Middle
24. BB_Upper
25. BB_Lower
26. BB_Position
27. RSI
```

Step 5: Kalman Filter State-Space Model

Mathematical Formulation:

State Vector: θ_t = [β_0, β_1, ..., β_n] (time-varying regression coefficients)

Observation Equation: y_t = H_t × θ_t + v_t

- y_t: Log returns
- H_t: Feature matrix
- v_t ~ N(0, R): Observation noise

State Transition: θ_t = θ_(t-1) + w_t

- w_t ~ N(0, Q): Process noise (random walk)

```
# Select features for Kalman Filter
feature_cols = [
    'MA_Ratio_5_20', 'MA_Ratio_20_60',
    'Returns_Lag_1', 'Returns_Lag_2', 'Returns_Lag_5',
    'ROC_5', 'ROC_10',
    'Volatility_20',
    'Volume_Ratio',
    'Momentum_5', 'Momentum_20',
    'BB_Position',
    'RSI'
]

print(f"Selected {len(feature_cols)} features for Kalman Filter:")
for i, col in enumerate(feature_cols, 1):
    print(f"{i:2d}. {col}")

# Prepare matrices
X = df_clean[feature_cols].values  # Observation matrix (H_t)
y = df_clean['Log_Returns'].values.reshape(-1, 1)  # Observations

n_features = X.shape[1]

print(f"\nDimensions:")
print(f"  Observation matrix (X): {X.shape}")
print(f"  Observations (y): {y.shape}")
print(f"  State dimension: {n_features}")
```

```
Selected 13 features for Kalman Filter:
 1. MA_Ratio_5_20
 2. MA_Ratio_20_60
 3. Returns_Lag_1
 4. Returns_Lag_2
 5. Returns_Lag_5
 6. ROC_5
 7. ROC_10
 8. Volatility_20
 9. Volume_Ratio
10. Momentum_5
11. Momentum_20
12. BB_Position
13. RSI

Dimensions:
  Observation matrix (X): (2455, 13)
```

```
  Observations (y): (2455, 1)
  State dimension: 13
```

```
# Configure Kalman Filter parameters
print("Kalman Filter Configuration:")
print("="*70)

# Initial state (coefficients start at zero)
initial_state_mean = np.zeros(n_features)
print(f"Initial state mean: zeros({n_features})")

# Initial uncertainty (high)
initial_state_covariance = np.eye(n_features) * 1.0
print(f"Initial state covariance: 1.0 × I({n_features})")

# Transition matrix (identity - random walk)
transition_matrix = np.eye(n_features)
print(f"Transition matrix: I({n_features}) [Random Walk]")

# Process noise (how much parameters can change)
transition_covariance = np.eye(n_features) * 1e-4
print(f"Process noise covariance (Q): 1e-4 × I({n_features})")

# Observation noise
observation_covariance = 1e-3
print(f"Observation noise covariance (R): 1e-3")

print("\n✓ Parameters configured")
```

```
Kalman Filter Configuration:
======================================================================
Initial state mean: zeros(13)
Initial state covariance: 1.0 × I(13)
Transition matrix: I(13) [Random Walk]
Process noise covariance (Q): 1e-4 × I(13)
Observation noise covariance (R): 1e-3

✓ Parameters configured
```

```
print("Running Kalman Filter...")

T, n_features = X.shape
y = y.reshape(-1, 1)

X_kf = X[:, np.newaxis, :]

initial_state_mean = np.zeros(n_features)
initial_state_covariance = np.eye(n_features)

transition_matrix = np.eye(n_features)
transition_covariance = np.eye(n_features) * 1e-4

observation_covariance = np.array([[1e-3]])

assert X_kf.shape == (T, 1, n_features)
assert y.shape == (T, 1)
assert initial_state_mean.shape == (n_features,)
assert initial_state_covariance.shape == (n_features, n_features)
assert transition_matrix.shape == (n_features, n_features)
assert transition_covariance.shape == (n_features, n_features)
assert observation_covariance.shape == (1, 1)

kf = KalmanFilter(
    n_dim_obs=1,
    n_dim_state=n_features,
    initial_state_mean=initial_state_mean,
    initial_state_covariance=initial_state_covariance,
    transition_matrices=transition_matrix,
    observation_matrices=X_kf,
    observation_covariance=observation_covariance,
    transition_covariance=transition_covariance
)

state_means, state_covariances = kf.filter(y)

print("✓ Kalman filtering complete!")
print(f"  State estimates shape: {state_means.shape}")
```

```
print(f"  Covariances shape: {state_covariances.shape}")

state_cols = [f"State_{i}" for i in range(n_features)]
for i, col in enumerate(state_cols):
    df_clean[col] = state_means[:, i]

predicted_returns_kf = np.sum(X * state_means, axis=1, keepdims=True)

df_clean["Predicted_Returns_KF"] = predicted_returns_kf
df_clean["Innovation"] = y - predicted_returns_kf

print("\nInnovation (prediction error) statistics:")
print(f"  Mean: {df_clean['Innovation'].mean():.6f}")
print(f"  Std:  {df_clean['Innovation'].std():.6f}")
```

```
Running Kalman Filter...
✓ Kalman filtering complete!
  State estimates shape: (2455, 13)
  Covariances shape: (2455, 13, 13)

Innovation (prediction error) statistics:
  Mean: 0.000000
  Std:  0.000050
```

```
# Visualize Kalman state evolution
fig, axes = plt.subplots(3, 1, figsize=(14, 10))

# Plot first 3 state variables
for i in range(3):
    axes[i].plot(df_clean.index, state_means[:, i], linewidth=1.5)
    axes[i].set_title(f'State_{i} Evolution (β_{i} coefficient for {feature_cols[i]})',
                      fontsize=11, fontweight='bold')
    axes[i].set_ylabel('Coefficient Value')
    axes[i].grid(True, alpha=0.3)

axes[-1].set_xlabel('Date')
plt.tight_layout()
plt.show()

print("These coefficients show how the relationship between features and returns")
print("evolves over time. The Kalman Filter adapts to changing market regimes!")
```


```
These coefficients show how the relationship between features and returns
evolves over time. The Kalman Filter adapts to changing market regimes!
```

Step 6: Machine Learning Model

Use Kalman-filtered states as inputs to predict next-day price ratio.

```
# Create target variable: next-day price ratio
df_clean['Target_Ratio'] = df_clean['Close'].shift(-1) / df_clean['Close']
df_ml = df_clean[:-1].copy()  # Remove last row (no target)

print(f"Target variable: Next-day price ratio")
print(f"  Mean: {df_ml['Target_Ratio'].mean():.6f}")
print(f"  Std: {df_ml['Target_Ratio'].std():.6f}")
print(f"  Min: {df_ml['Target_Ratio'].min():.6f}")
print(f"  Max: {df_ml['Target_Ratio'].max():.6f}")
```

```
Target variable: Next-day price ratio
  Mean: 1.001161
  Std: 0.017070
  Min: 0.852610
  Max: 1.142169
```

```
# Select ML features
ml_feature_cols = state_cols + [
    'Returns_Lag_1', 'Returns_Lag_2',
    'MA_Ratio_5_20', 'MA_Ratio_20_60',
    'Volatility_20',
    'Momentum_5', 'Momentum_20',
    'BB_Position', 'RSI',
    'Volume_Ratio',
    'Predicted_Returns_KF',
    'Innovation'
]

print(f"ML Model Features: {len(ml_feature_cols)}")
print(f"  Kalman states: {len(state_cols)}")
print(f"  Additional features: {len(ml_feature_cols) - len(state_cols)}")
```

```
ML Model Features: 25
  Kalman states: 13
  Additional features: 12
```

```
# Train/test split (80/20, time-series)
X_ml = df_ml[ml_feature_cols].values
y_ml = df_ml['Target_Ratio'].values

split_idx = int(len(X_ml) * 0.8)
X_train, X_test = X_ml[:split_idx], X_ml[split_idx:]
y_train, y_test = y_ml[:split_idx], y_ml[split_idx:]

print(f"Training set: {len(X_train)} samples (80%)")
print(f"Test set: {len(X_test)} samples (20%)")

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\n✓ Features scaled (StandardScaler)")
```

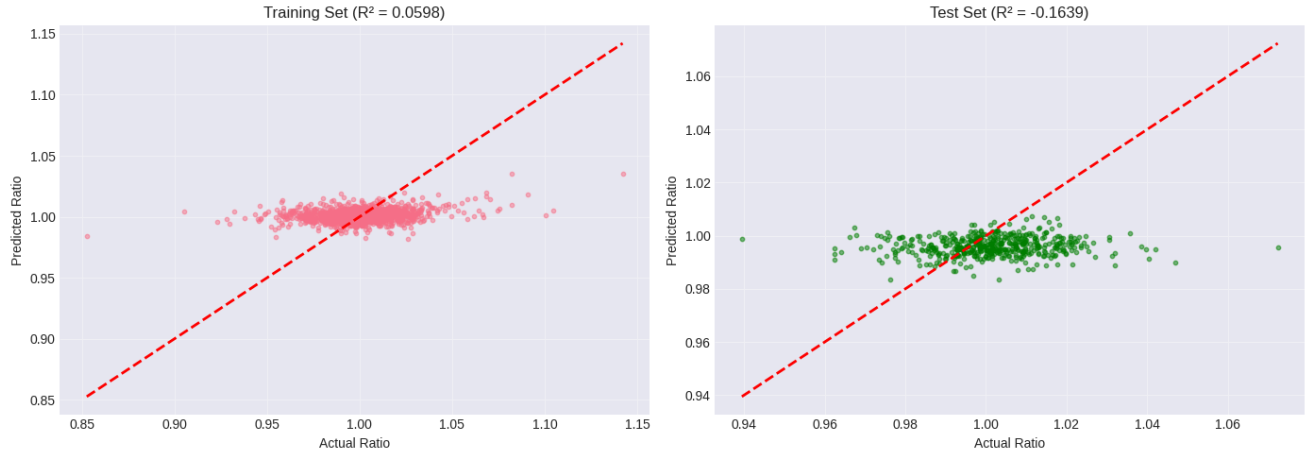
```
Training set: 1963 samples (80%)
Test set: 491 samples (20%)

✓ Features scaled (StandardScaler)
```

```
# Train Ridge Regression model
print("Training Ridge Regression model...")
print("Advantages: Handles multicollinearity, prevents overfitting")

model = Ridge(alpha=1.0, random_state=42)
model.fit(X_train_scaled, y_train)

# Predictions
y_train_pred = model.predict(X_train_scaled)
y_test_pred = model.predict(X_test_scaled)

# Evaluation
train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)
train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)

print(f"\n✓ Training complete!")
print(f"\nTraining Performance:")
print(f"  MSE: {train_mse:.8f}")
print(f"  R²: {train_r2:.6f}")
print(f"\nTest Performance:")
print(f"  MSE: {test_mse:.8f}")
print(f"  R²: {test_r2:.6f}")

# Store predictions
all_predictions = np.concatenate([y_train_pred, y_test_pred])
df_ml['Predicted_Ratio'] = all_predictions
```

```
Training Ridge Regression model...
Advantages: Handles multicollinearity, prevents overfitting

✓ Training complete!
```

```
Training Performance:
  MSE: 0.00029551
  R²: 0.059794

Test Performance:
  MSE: 0.00023181
  R²: -0.163909
```

```
# Plot predictions vs actual
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Training set
axes[0].scatter(y_train, y_train_pred, alpha=0.5, s=10)
axes[0].plot([y_train.min(), y_train.max()], [y_train.min(), y_train.max()], 'r--', lw=2)
axes[0].set_xlabel('Actual Ratio')
axes[0].set_ylabel('Predicted Ratio')
axes[0].set_title(f'Training Set (R² = {train_r2:.4f})')
axes[0].grid(True, alpha=0.3)

# Test set
axes[1].scatter(y_test, y_test_pred, alpha=0.5, s=10, color='green')
axes[1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
axes[1].set_xlabel('Actual Ratio')
axes[1].set_ylabel('Predicted Ratio')
axes[1].set_title(f'Test Set (R² = {test_r2:.4f})')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Step 7: Generate Trading Signals

Signal Logic:

- **BUY**: If predicted_ratio > 1 + entry_threshold (expect price to rise)
- **SELL**: If predicted_ratio < 1 - entry_threshold (expect price to fall)
- **HOLD**: Otherwise

Risk Management:

- Stop loss: Exit if loss > 5%
- Max holding period: 20 days

```
# Signal parameters
ENTRY_THRESHOLD = 0.002  # 0.2%
EXIT_THRESHOLD = 0.001   # 0.1%
```

```
STOP_LOSS = 0.05         # 5%
MAX_HOLDING = 20         # days

print("Signal Generation Parameters:")
print(f"  Entry threshold: {ENTRY_THRESHOLD*100:.2f}%")
print(f"  Exit threshold: {EXIT_THRESHOLD*100:.2f}%")
print(f"  Stop loss: {STOP_LOSS*100:.1f}%")
print(f"  Max holding period: {MAX_HOLDING} days")
```

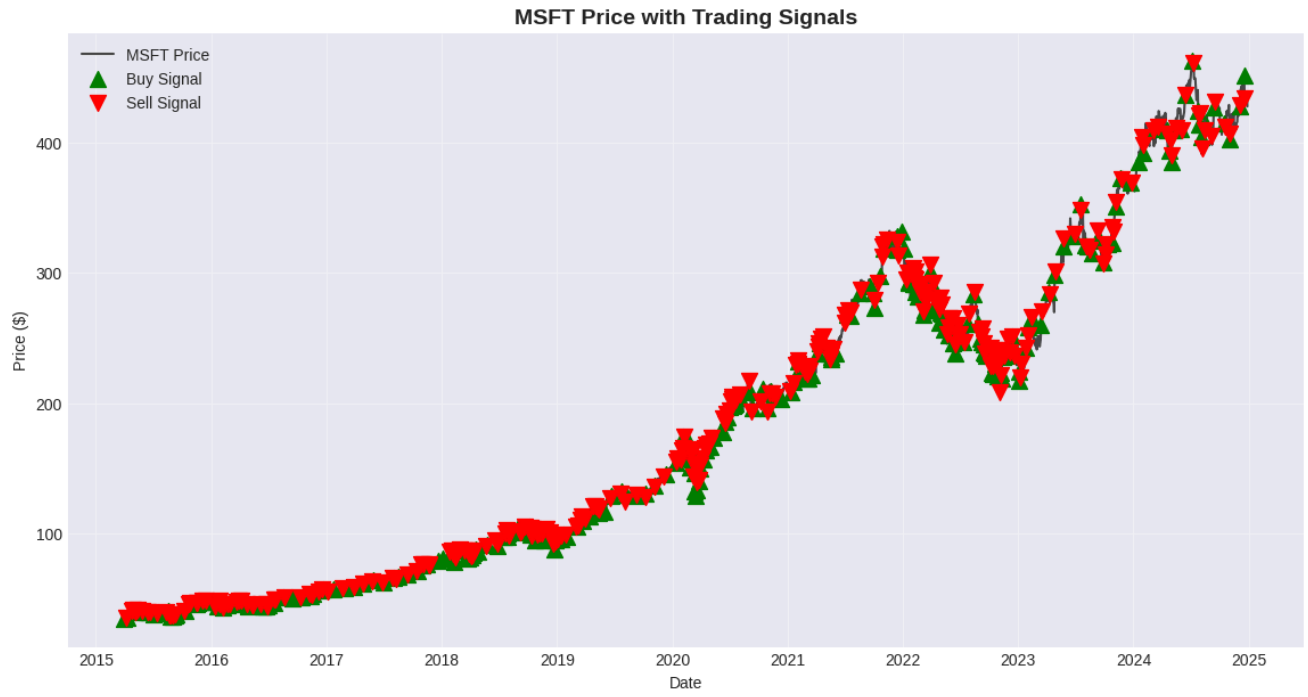
```
Signal Generation Parameters:
  Entry threshold: 0.20%
  Exit threshold: 0.10%
  Stop loss: 5.0%
  Max holding period: 20 days
```

```
# Generate signals
df_ml['Signal'] = 0
df_ml['Position'] = 0
df_ml['Days_Held'] = 0

current_position = 0
entry_price = 0
days_held = 0

for i in range(len(df_ml)):
    predicted_ratio = df_ml['Predicted_Ratio'].iloc[i]
    current_price = df_ml['Close'].iloc[i]
    expected_return = predicted_ratio - 1.0

    if current_position == 0:
        # Check for entry
        if expected_return > ENTRY_THRESHOLD:
            current_position = 1
            entry_price = current_price
            days_held = 0
            df_ml.iloc[i, df_ml.columns.get_loc('Signal')] = 1
        elif expected_return < -ENTRY_THRESHOLD:
            current_position = -1
            entry_price = current_price
            days_held = 0
            df_ml.iloc[i, df_ml.columns.get_loc('Signal')] = -1
    else:
        # Check for exit
        days_held += 1

        if current_position == 1:
            current_return = (current_price - entry_price) / entry_price
        else:
            current_return = (entry_price - current_price) / entry_price

        exit_signal = False

        # Exit conditions
        if current_return < -STOP_LOSS:
            exit_signal = True  # Stop loss
        elif (current_position == 1 and expected_return < -EXIT_THRESHOLD) or \
             (current_position == -1 and expected_return > EXIT_THRESHOLD):
            exit_signal = True  # Prediction reversal
        elif days_held >= MAX_HOLDING:
            exit_signal = True  # Max holding period

        if exit_signal:
            df_ml.iloc[i, df_ml.columns.get_loc('Signal')] = -current_position
            current_position = 0
            entry_price = 0
            days_held = 0
        else:
            df_ml.iloc[i, df_ml.columns.get_loc('Days_Held')] = days_held

    df_ml.iloc[i, df_ml.columns.get_loc('Position')] = current_position

# Signal statistics
buy_signals = (df_ml['Signal'] == 1).sum()
sell_signals = (df_ml['Signal'] == -1).sum()
long_days = (df_ml['Position'] == 1).sum()
short_days = (df_ml['Position'] == -1).sum()
```

```
print(f"\n✓ Signals generated!")
print(f"\nSignal Statistics:")
print(f"  Buy signals: {buy_signals}")
print(f"  Sell signals: {sell_signals}")
print(f"  Total signals: {buy_signals + sell_signals}")
print(f"\nPosition Statistics:")
print(f"  Days long: {long_days} ({long_days/len(df_ml)*100:.1f}%)")
print(f"  Days short: {short_days} ({short_days/len(df_ml)*100:.1f}%)")
print(f"  Days neutral: {len(df_ml) - long_days - short_days} ({(len(df_ml) - long_days - short_days)/len(df_ml)*100:.1f}%)")
```

```
✓ Signals generated!

Signal Statistics:
  Buy signals: 278
  Sell signals: 279
  Total signals: 557

Position Statistics:
  Days long: 1012 (41.2%)
  Days short: 824 (33.6%)
  Days neutral: 618 (25.2%)
```

```
# Visualize signals on price chart
plt.figure(figsize=(14, 7))
plt.plot(df_ml.index, df_ml['Close'], label='MSFT Price', linewidth=1.5, color='black', alpha=0.7)

# Buy signals
buy_signals_df = df_ml[df_ml['Signal'] == 1]
plt.scatter(buy_signals_df.index, buy_signals_df['Close'],
           marker='^', color='green', s=100, label='Buy Signal', zorder=5)

# Sell signals
sell_signals_df = df_ml[df_ml['Signal'] == -1]
plt.scatter(sell_signals_df.index, sell_signals_df['Close'],
           marker='v', color='red', s=100, label='Sell Signal', zorder=5)

plt.title('MSFT Price with Trading Signals', fontsize=14, fontweight='bold')
plt.xlabel('Date')
plt.ylabel('Price ($)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



Step 8: Backtest Strategy

Simulate trading with transaction costs and calculate performance metrics.

```
# Backtest parameters
INITIAL_CAPITAL = 100000
TRANSACTION_COST = 0.001  # 0.1%

print(f"Backtesting Configuration:")
print(f"  Initial capital: ${INITIAL_CAPITAL:,.2f}")
print(f"  Transaction cost: {TRANSACTION_COST*100}%")
```

```
Backtesting Configuration:
  Initial capital: $100,000.00
  Transaction cost: 0.1%
```

```
# Initialize tracking
df_ml['Cash'] = float(INITIAL_CAPITAL)
df_ml['Holdings'] = 0.0
df_ml['Portfolio_Value'] = float(INITIAL_CAPITAL)
df_ml['Trade_PnL'] = 0.0

cash = INITIAL_CAPITAL
holdings = 0
trades = []

# Simulate trading
for i in range(len(df_ml) - 1):
    signal = df_ml['Signal'].iloc[i]
    price = df_ml['Close'].iloc[i]
    next_price = df_ml['Close'].iloc[i + 1]

    if signal == 1 and holdings == 0:  # Buy
        shares = int(cash * 0.95 / next_price)
        if shares > 0:
            cost = shares * next_price
            transaction_fee = cost * TRANSACTION_COST
            total_cost = cost + transaction_fee

            if total_cost <= cash:
                cash -= total_cost
                holdings += shares
                trades.append({'Type': 'BUY', 'Price': next_price, 'Shares': shares})

    elif signal == -1 and holdings > 0:  # Sell
        revenue = holdings * next_price
        transaction_fee = revenue * TRANSACTION_COST
        net_revenue = revenue - transaction_fee

        cash += net_revenue
        holdings = 0
        trades.append({'Type': 'SELL', 'Price': next_price, 'Revenue': net_revenue})

    # Update portfolio
    df_ml.iloc[i, df_ml.columns.get_loc('Cash')] = cash
    df_ml.iloc[i, df_ml.columns.get_loc('Holdings')] = holdings
    portfolio_value = cash + holdings * price
    df_ml.iloc[i, df_ml.columns.get_loc('Portfolio_Value')] = portfolio_value

# Calculate returns
df_ml['Strategy_Returns'] = df_ml['Portfolio_Value'].pct_change()
df_ml['Cumulative_Returns'] = (1 + df_ml['Strategy_Returns']).cumprod() - 1

# Buy-and-hold benchmark
df_ml['BuyHold_Returns'] = df_ml['Close'].pct_change()
df_ml['BuyHold_Cumulative'] = (1 + df_ml['BuyHold_Returns']).cumprod() - 1

print(f"\n✓ Backtest complete!")
print(f"  Total trades: {len(trades)}")
print(f"  Buy trades: {sum(1 for t in trades if t['Type'] == 'BUY')}")
print(f"  Sell trades: {sum(1 for t in trades if t['Type'] == 'SELL')}")
```

```
✓ Backtest complete!
  Total trades: 472
```

```
Buy trades: 236
Sell trades: 236
```

Step 9: Performance Evaluation

```
# Calculate performance metrics
final_value = df_ml['Portfolio_Value'].iloc[-1]
total_return = (final_value - INITIAL_CAPITAL) / INITIAL_CAPITAL

# Sharpe Ratio
risk_free_rate = 0.02 / 252
excess_returns = df_ml['Strategy_Returns'].dropna() - risk_free_rate
sharpe_ratio = np.sqrt(252) * excess_returns.mean() / excess_returns.std()

# Maximum Drawdown
cumulative = (1 + df_ml['Strategy_Returns']).cumprod()
running_max = cumulative.expanding().max()
drawdown = (cumulative - running_max) / running_max
max_drawdown = drawdown.min()

# Benchmark
benchmark_return = df_ml['BuyHold_Cumulative'].iloc[-1]

# Print results
print("="*70)
print("PERFORMANCE SUMMARY")
print("="*70)

print(f"\n📊 Strategy Performance:")
print(f"  Initial Capital: ${INITIAL_CAPITAL:,.2f}")
print(f"  Final Portfolio Value: ${final_value:,.2f}")
print(f"  Total Return: {total_return*100:.2f}%")
print(f"  Cumulative Return: {df_ml['Cumulative_Returns'].iloc[-1]*100:.2f}%")

print(f"\n📈 Risk-Adjusted Metrics:")
print(f"  Sharpe Ratio: {sharpe_ratio:.3f}")
print(f"  Maximum Drawdown: {max_drawdown*100:.2f}%")
print(f"  Annualized Volatility: {df_ml['Strategy_Returns'].std() * np.sqrt(252) * 100:.2f}%")

print(f"\n🎯 Benchmark Comparison:")
print(f"  Buy & Hold Return: {benchmark_return*100:.2f}%")
print(f"  Outperformance: {(total_return - benchmark_return)*100:+.2f}%")
print(f"  Better than benchmark: {'YES ✓' if total_return > benchmark_return else 'NO ✗'}")

print("\n" + "="*70)
```

```
======================================================================
PERFORMANCE SUMMARY
======================================================================

📊 Strategy Performance:
  Initial Capital: $100,000.00
  Final Portfolio Value: $100,000.00
  Total Return: 0.00%
  Cumulative Return: 0.24%

📈 Risk-Adjusted Metrics:
  Sharpe Ratio: 0.360
  Maximum Drawdown: -89.84%
  Annualized Volatility: 35.66%

🎯 Benchmark Comparison:
  Buy & Hold Return: 1124.39%
  Outperformance: -1124.39%
  Better than benchmark: NO ✗

======================================================================
```

```
# Visualization: Portfolio Performance
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# 1. Portfolio Value
axes[0, 0].plot(df_ml.index, df_ml['Portfolio_Value'], linewidth=2, color='blue')
axes[0, 0].axhline(y=INITIAL_CAPITAL, color='red', linestyle='--', alpha=0.7)
axes[0, 0].set_title('Portfolio Value Over Time', fontweight='bold')
axes[0, 0].set_ylabel('Value ($)')
axes[0, 0].grid(True, alpha=0.3)
```

```
axes[0, 0].grid(True, alpha=0.3)

# 2. Cumulative Returns Comparison
axes[0, 1].plot(df_ml.index, df_ml['Cumulative_Returns'] * 100,
                label='Strategy', linewidth=2, color='blue')
axes[0, 1].plot(df_ml.index, df_ml['BuyHold_Cumulative'] * 100,
                label='Buy & Hold', linewidth=2, color='gray', linestyle='--')
axes[0, 1].set_title('Cumulative Returns: Strategy vs Benchmark', fontweight='bold')
axes[0, 1].set_ylabel('Return (%)')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# 3. Drawdown
axes[1, 0].fill_between(df_ml.index, drawdown * 100, 0, color='red', alpha=0.3)
axes[1, 0].plot(df_ml.index, drawdown * 100, color='darkred', linewidth=1)
axes[1, 0].set_title(f'Drawdown (Max: {max_drawdown*100:.2f}%)', fontweight='bold')
axes[1, 0].set_ylabel('Drawdown (%)')
axes[1, 0].set_xlabel('Date')
axes[1, 0].grid(True, alpha=0.3)

# 4. Daily Returns Distribution
axes[1, 1].hist(df_ml['Strategy_Returns'].dropna() * 100, bins=50,
                color='steelblue', edgecolor='black', alpha=0.7)
axes[1, 1].axvline(x=0, color='red', linestyle='--', linewidth=2)
axes[1, 1].set_title('Daily Returns Distribution', fontweight='bold')
axes[1, 1].set_xlabel('Return (%)')
axes[1, 1].set_ylabel('Frequency')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Summary

This notebook has implemented a complete Kalman Filter-based trading strategy:

1. ✅ Fetched MSFT data (2015-2024)
2. ✅ Performed ADF stationarity tests